



PROYECTO SGE

CFGS Desarrollo de Aplicaciones
Multiplataforma
Informática y Comunicaciones

<Práctica 11 Modulo ODOO>

Año: <2025>

Fecha de presentación: (28/12/2025)

Nombre y Apellidos: Yubo Lin

Email: yubo.lin@educa.jcyl.es

Contenido

1	Introducción.....	3
2	Estado del arte Definición de ERP	3
3	Descripción general del proyecto	4
3.1	Objetivos	4
•	5 modelos de datos relacionados.....	4
3.2	Entorno de trabajo	4
4	Documentación técnica.....	7
4.1	Análisis del sistema.....	7
4.2	Diseño de la base de datos.....	7
4.3	Implementación.....	8
4.4	Pruebas de funcionamiento	18
4.5	Despliegue de la aplicación	20
5	Manuales.....	21
5.1	Manual de usuario	21
5.2	Manual de instalación	26
6	Conclusiones y posibles ampliaciones	26
7	Bibliografía	27

1 Introducción

- El proyecto desarrolla un módulo personalizado para Odoo llamado “Library Yubo”, orientado a la gestión integral de una biblioteca. Permite administrar recursos como libros, entidades como autores y géneros, y usuarios como socios, facilitando un control eficiente de la información.
- Se realiza dentro del módulo de Sistemas de Gestión Empresarial (SGE) con el objetivo de adquirir experiencia en desarrollo para Odoo, uso de su ORM y creación de servicios API REST.
- La memoria del proyecto recoge todo el ciclo de vida del software: estado del arte, requisitos, diseño de base de datos, implementación, pruebas y manuales de usuario.

2 Estado del arte Definición de ERP

- Definición de ERP Un ERP (Enterprise Resource Planning) es un sistema de software integrado que permite a las empresas gestionar sus procesos de negocio principales (contabilidad, ventas, inventario) en una única plataforma.
- ¿Por qué Odoo?

Se ha elegido Odoo por ser el ERP de código abierto líder en el mercado. Su arquitectura modular basada en Python y PostgreSQL permite una gran flexibilidad para desarrollar soluciones personalizadas como esta biblioteca.

3 Descripción general del proyecto

3.1 Objetivos

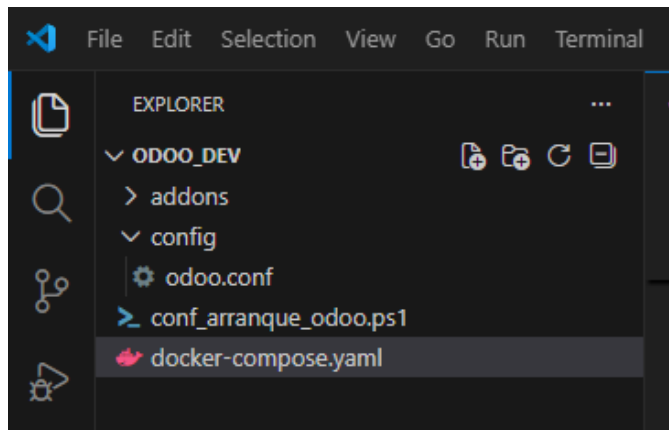
El objetivo técnico es crear un módulo instalable que incluya:

- 5 modelos de datos relacionados.
- Vistas avanzadas (Kanban, Tree, Form).
- Lógica de negocio automatizada (Cálculos de fechas, IDs automáticos).
- Interfaz API para conexión externa.

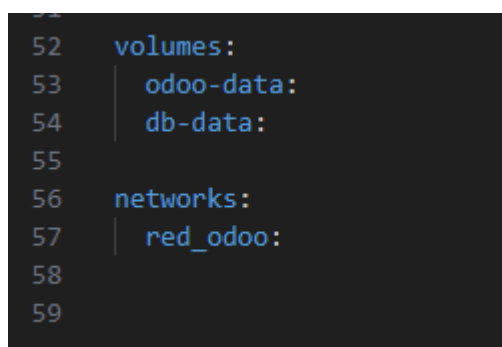
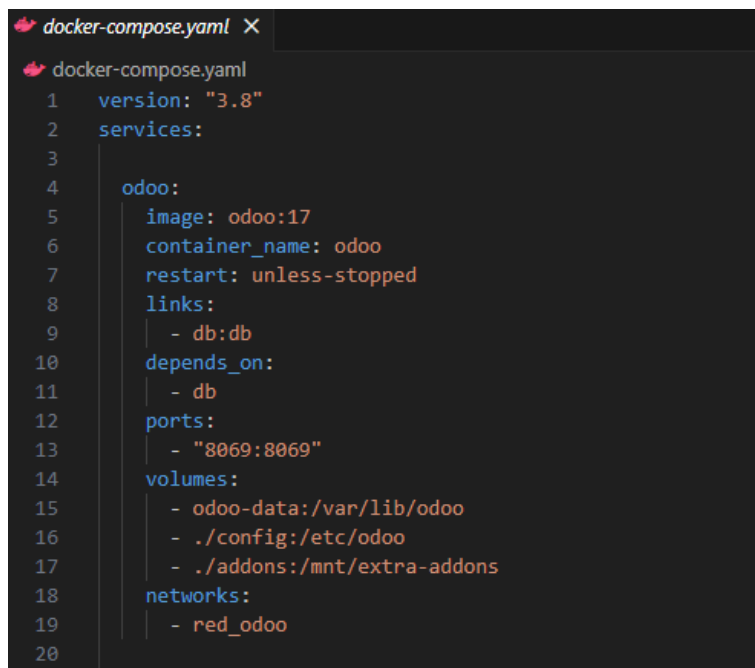
3.2 Entorno de trabajo

- Docker: Para la contenerización de Odoo y PostgreSQL.
- Visual Studio Code: Como entorno de desarrollo (IDE).
- Postman/Navegador: Para las pruebas de API.

Uso Visual Studio para escribir código de Python , así como para la gestión de archivos de configuración y vistas XML de Odoo.



docker-compose.yaml



Aqui configurado base de dato con PostgreSQL

```
db:
  image: postgres:latest
  container_name: container-postgresdb
  restart: unless-stopped
  environment:
    - DATABASE_HOST=127.0.0.1
    - POSTGRES_DB=postgres
    - POSTGRES_PASSWORD=odoo
    - POSTGRES_USER=odoo
    - PGDATA=/var/lib/postgresql/data/pgdata
  volumes:
    - db-data:/var/lib/postgresql/data
  networks:
    - red_odoo
  ports:
    - "5432:5432"
```

Para la gestión y visualización de la base de datos **PostgreSQL**, se ha integrado **pgAdmin4** mediante un contenedor Docker

```
pgadmin:
  image: dpage/pgadmin4:latest
  depends_on:
    - db
  ports:
    - "80:80"
  environment:
    PGADMIN_DEFAULT_EMAIL: pgadmin4@pgadmin.org
    PGADMIN_DEFAULT_PASSWORD: admin
  restart: unless-stopped
  networks:
    - red_odoo
```

4 Documentación técnica

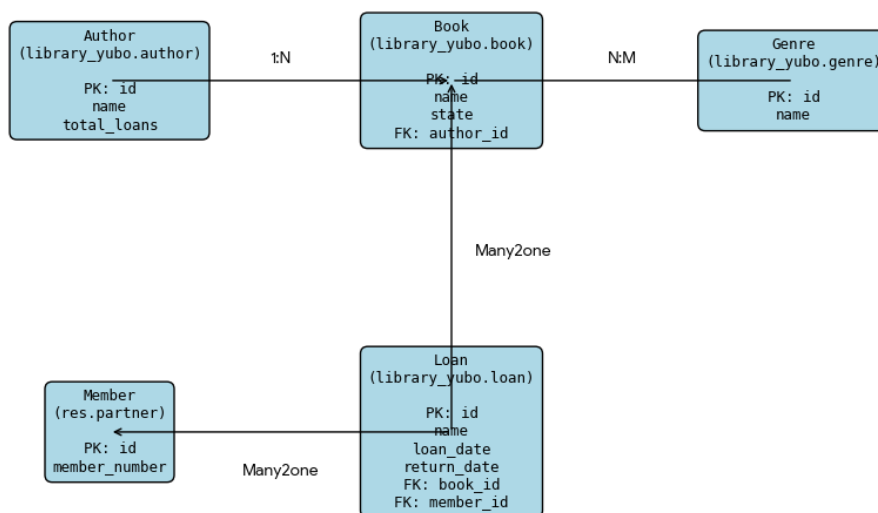
4.1 Análisis del sistema

El sistema se basa en la arquitectura Modelo-Vista-Controlador. Los usuarios del grupo "Library / User" tienen permisos de acceso (ACL) definidos en ir.model.access.csv para crear, leer, escribir y eliminar registros en los modelos principales.

```
security > ir.model.access.csv
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_library_yubo_book,library_yubo_book,model_library_yubo_book,base.group_user,1,1,1,1
3 access_library_yubo_author,library_yubo_author,model_library_yubo_author,base.group_user,1,1,1,1
4 access_library_yubo_genre,library_yubo_genre,model_library_yubo_genre,base.group_user,1,1,1,1
5 access_library_yubo_loan,library_yubo_loan,model_library_yubo_loan,base.group_user,1,1,1,1
```

4.2 Diseño de la base de datos

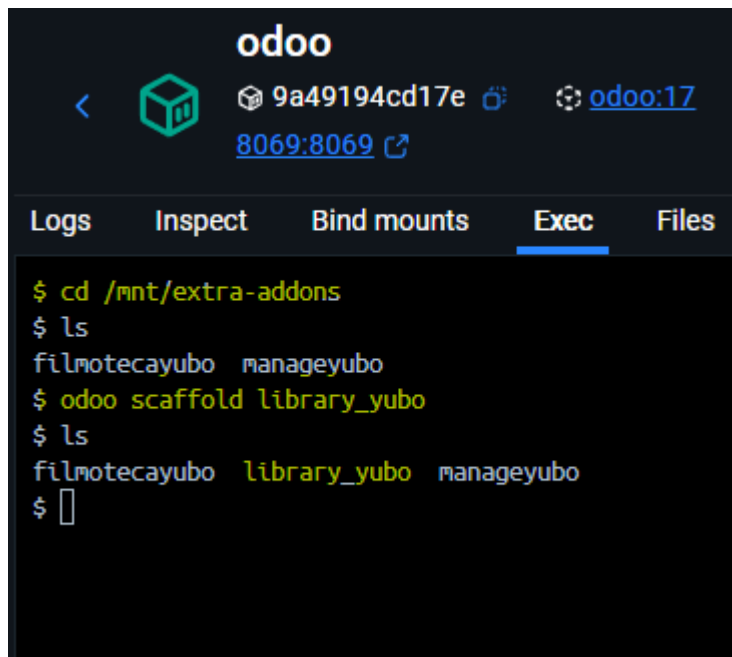
Diagrama Entidad-Relación: Library Yubo



4.3 Implementación

Scaffold

Generó automáticamente la estructura básica de directorios del módulo



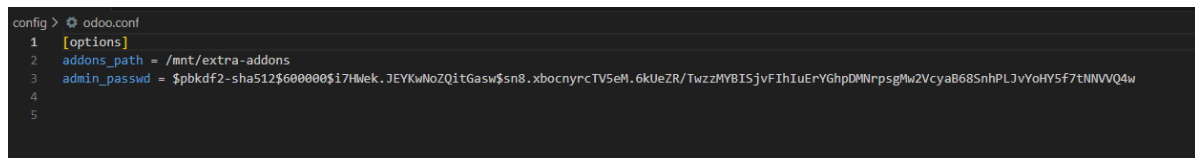
The screenshot shows the Odoo Docker container interface. The 'Exec' tab is active, displaying a terminal session. The user has navigated to the directory `/mnt/extra-addons` and executed the command `odoo scaffold library_yubo`. The output shows the directory listing before and after the command, confirming the creation of the `library_yubo` directory.

```

$ cd /mnt/extra-addons
$ ls
filmotecayubo  manageyubo
$ odoo scaffold library_yubo
$ ls
filmotecayubo  library_yubo  manageyubo
$ 

```

lo que garantiza que Odoo pueda reconocer el módulo de biblioteca creado en `/mnt/extra-addons`.



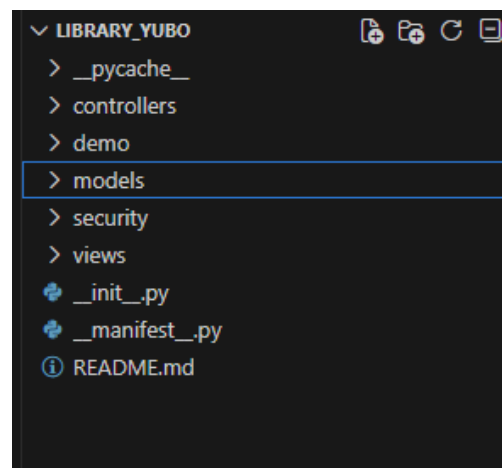
The screenshot shows the `odoo.conf` file in the configuration editor. The `addons_path` is set to `/mnt/extra-addons`, which allows Odoo to recognize modules located in that directory.

```

1  [options]
2  addons_path = /mnt/extra-addons
3  admin_passwd = $pbkdf2-sha512$600000$17HwEk.JEYKwNoZQitGasw$sn8.xbocnyrcTVSeH.6kUeZR/TwzzMYBISjvFIhIuErYGhpDMNrpsgMw2VcyaB68SnhPLJvYohY5f7tNNVQ4w
4
5

```

Al final estructura era así



Parte Modelos

Todos los modelos están en un solo archivo llamado models.py.

Herencia del modelo, definido para viene a leer o pedir prestados libros

```
models > models.py > Author
1  # -*- coding: utf-8 -*-
2
3  from odoo import models, fields, api
4  from odoo.exceptions import ValidationError
5  from datetime import timedelta
6
7  # 1. Herencia del modelo: hereda del modelo existente de lector / partner
8  class Member(models.Model):
9      _inherit = 'res.partner'
10
11      member_number = fields.Char(string='Número de Socio')
12
```

Clase Autor

`_get_total_loans` para saber

Cuántas veces en total se han prestado todos los libros de este autor

```
13 # 2. Modelo de Autor
14 class Author(models.Model):
15     _name = 'library_yubo.author'
16     _description = 'Autores de libros'
17
18     name = fields.Char(string='Nombre', required=True)
19     # Relación One2many: un autor puede tener varios libros
20     book_ids = fields.One2many('library_yubo.book', 'author_id', string='Libros')
21
22
23     total_loans = fields.Integer(string="Total Préstamos", compute="_get_total_loans")
24
25     def _get_total_loans(self):
26         # Utilice env para obtener el modelo de préstamo
27         Loan = self.env['library_yubo.loan']
28         for author in self:
29             # Busque todos los libros de este autor
30             # Cuente cuántas veces se han tomado prestados estos libros
31             count = Loan.search_count([('book_id.author_id', '=', author.id)])
32             author.total_loans = count
33
```

Clase Libro

```
34 # 3. Modelo de Libro (versión actualizada)
35 class Book(models.Model):
36     _name = 'library_yubo.book'
37     _description = 'Libros de la biblioteca'
38
39     name = fields.Char(string='Título', required=True)
40     # Relación Many2one: varios libros corresponden a un autor
41     author_id = fields.Many2one('library_yubo.author', string='Autor')
42     # Relación Many2many: un libro puede pertenecer a varios géneros
43     genre_ids = fields.Many2many('library_yubo.genre', string='Géneros')
44
45     state = fields.Selection([
46         ('available', 'Disponible'),
47         ('borrowed', 'Prestado')
48     ], string='Estado', default='available')
49
```

Clase género de libro

```
50 # 4. Modelo de Género de Libro
51 class Genre(models.Model):
52     _name = 'library_yubo.genre'
53     _description = 'Géneros de libros'
54
55     name = fields.Char(string='Género', required=True)
56
```

Clase Préstamos

```
57 # 5. Modelo de Préstamo
58 class Loan(models.Model):
59     _name = 'library_yubo.loan'
60     _description = 'Registro de préstamos'
61
62     book_id = fields.Many2one('library_yubo.book', string='Libro', required=True)
63     member_id = fields.Many2one('res.partner', string='Socio', required=True)
64
65
66     author_id = fields.Many2one(
67         'library_yubo.author',
68         string="Autor",
69         related='book_id.author_id', # Encuentra author_id usando book_id
70         readonly=True,
71         store=True
72     )
73
74     # Uso de lambda para establecer el valor por defecto
75     loan_date = fields.Date(
76         string='Fecha de Préstamo',
77         default=lambda self: fields.Date.today()
78     )
79     return_date = fields.Date(string='Fecha de Devolución')
80
```

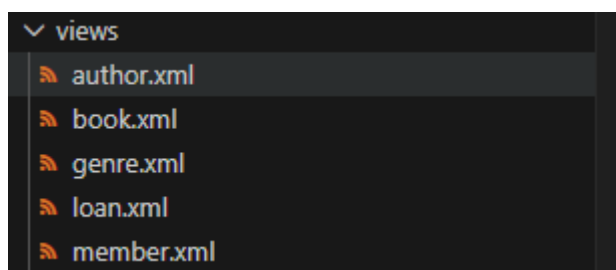
En el modelo de préstamos se implementan campos calculados y restricciones para automatizar la duración del préstamo, validar fechas y generar una referencia única del préstamo

```

80
81     # Campo calculado avanzado (almacenado en la base de datos)
82     duration = fields.Integer(
83         string='Días de Préstamo',
84         compute='_compute_duration',
85         store=True # <--- Significa que se almacena en una base de datos.
86     )
87
88     # Decorador @api.depends
89     @api.depends('loan_date', 'return_date')
90     def _compute_duration(self):
91         for record in self:
92             if record.loan_date and record.return_date:
93                 record.duration = (record.return_date - record.loan_date).days
94             else:
95                 record.duration = 0
96
97     # Decorador @api.constrains (manejo de errores)
98     @api.constrains('return_date')
99     def _check_return_date(self):
100         for record in self:
101             if record.return_date and record.return_date < record.loan_date:
102                 raise ValidationError(
103                     'La fecha de devolución no puede ser anterior a la de préstamo.'
104                 )
105     name = fields.Char(string="Referencia Préstamo", compute="_get_loan_ref", store=True)
106
107     @api.depends('book_id', 'member_id')
108     def _get_loan_ref(self):
109         for loan in self:
110             if loan.book_id and loan.member_id:
111                 # genera id
112                 loan.name = f"{loan.book_id.name[:3].upper()}_{loan.member_id.name[:3].upper()}_{loan.id}"
113             else:
114                 loan.name = "NEW"
115
116

```

En la carpeta **Views** tienes un XML por cada modelo autor, libro, genero, prestamos, Socio



Author.xml , **Views** de los autores del libro

```
<odoo>
  <data>
    <record model="ir.ui.view" id="view_library_yubo_author_tree">
      <field name="name">library.yubo.author.tree</field>
      <field name="model">library_yubo.author</field>
      <field name="arch" type="xml">
        <tree>
          <field name="name"/>
          <field name="total_loans" string="Veces Prestado" sum="Total"/>
          <field name="book_ids" widget="many2many_tags"/>
        </tree>
      </field>
    </record>

    <record model="ir.ui.view" id="view_library_yubo_author_form">
      <field name="name">library.yubo.author.form</field>
      <field name="model">library_yubo.author</field>
      <field name="arch" type="xml">
        <form>
          <sheet>
            <group>
              <field name="name"/>
            </group>
            <notebook>
              <page string="Libros Publicados">
                <field name="book_ids" readonly="1"/>
              </page>
            </notebook>
          </sheet>
        </form>
      </field>
    </record>

    <record model="ir.actions.act_window" id="action_library_yubo_author">
      <field name="name">Autores</field>
      <field name="res_model">library_yubo.author</field>
      <field name="view_mode">tree,form</field>
    </record>

    <menuitem name="Autores" id="menu_library_yubo_author"
      parent="menu_library_yubo_management"
      action="action_library_yubo_author" sequence="30"/>
  </data>
</odoo>
```

Book.xml , raíz esta aquí , entonces en manifest.py tengo que configurar se carga primero, Kanban está aquí también

```

1 <odoo>
2 <data>
3 <!-- explicit list view definition -->
4
5 <record model="ir.ui.view" id="vista_library_yubo_book_tree">
6 <field name="name">vista_library_yubo_book_tree</field>
7 <field name="model">library_yubo.book</field>
8 <field name="arch" type="xml">
9 <tree>
10 <field name="name" />
11 <field name="author_id" />
12 <field name="state" />
13 </tree>
14 </field>
15 </record>
16
17 <record model="ir.ui.view" id="vista_library_yubo_book_kanban">
18 <field name="name">vista_library_yubo_book_kanban</field>
19 <field name="model">library_yubo.book</field>
20 <field name="arch" type="xml">
21 <kanban class="o_kanban_mobile">
22 <field name="name" />
23 <field name="author_id" />
24 <field name="state" />
25 <templates>
26 <t t-name="kanban-box">
27 <div t-attf-class="oe_kanban_global_click shadow-sm border-primary">
28 <div class="o_kanban_details">
29 <strong class="o_kanban_record_title" style="font-size: 16px;">
30 <field name="name" />
31 </strong>
32 <div class="text-muted"> Autor: <field name="author_id" />
33 </div>
34 <div class="mt-2">
35 <span>
36 <t-attf-class="badge #{{record.state.raw_value == 'available' ? 'bg-success' : 'bg-warning'}} text-white">
37 <field name="state" />
38 </span>
39 </div>
40 </div>
41 </div>
42 </t>
43 </templates>
44 </kanban>
45 </field>
46 </record>

```

```

47
48 <record model="ir.ui.view" id="vista_library_yubo_book_form">
49 <field name="name">vista_library_yubo_book_form</field>
50 <field name="model">library_yubo.book</field>
51 <field name="arch" type="xml">
52 <form string="formulario_book">
53 <sheet>
54 <group name="group_top">
55 <field name="name" />
56 <field name="state" />
57 <field name="author_id" />
58 <field name="genre_ids" />
59 </group>
60 </sheet>
61 </form>
62 </field>
63 </record>
64
65
66 <record model="ir.actions.act_window" id="accion_library_yubo_book_form">
67 <field name="name">Listado de Libro</field>
68 <field name="type">ir.actions.act_window</field>
69 <field name="res_model">library_yubo.book</field>
70 <field name="view_mode">tree,form,kanban</field>
71 <field name="help" type="html">
72 <p class="oe_view_nocontent_create">
73 Libro
74 </p>
75 <p> Click <strong> 'Crear' </strong> para añadir nuevos elementos </p>
76 </field>
77 </record>

```

Menú raíz y su parte

```

80      <!-- Menú raíz -->
81
82      <menuitem name="library_yubo" id="menu_library_yubo_raiz" />
83
84      <!-- menu categories -->
85
86      <menuitem name="Management" id="menu_library_yubo_management"
87        parent="menu_library_yubo_raiz" />
88
89
90      <!-- actions -->
91
92      <menuitem name="Libro" id="menu_library_yubo_book" parent="menu_library_yubo_management"
93        action="accion_library_yubo_book_form" />
94
95
96    </data>
97  </odoo>

```

Genre.xml, **Views** de los géneros del libro

```

views > genre.xml > odoo
1  <odoo>
2    <data>
3      <record model="ir.ui.view" id="view_library_yubo_genre_tree">
4        <field name="name">library.yubo.genre.tree</field>
5        <field name="model">library_yubo.genre</field>
6        <field name="arch" type="xml">
7          <tree>
8            <field name="name"/>
9          </tree>
10       </field>
11     </record>
12
13     <record model="ir.ui.view" id="view_library_yubo_genre_form">
14       <field name="name">library.yubo.genre.form</field>
15       <field name="model">library_yubo.genre</field>
16       <field name="arch" type="xml">
17         <form>
18           <sheet>
19             <group>
20               <field name="name"/>
21             </group>
22           </sheet>
23         </form>
24       </field>
25     </record>
26
27     <record model="ir.actions.act_window" id="action_library_yubo_genre">
28       <field name="name">Géneros</field>
29       <field name="res_model">library_yubo.genre</field>
30       <field name="view_mode">tree,form</field>
31     </record>
32
33     <menuitem name="Géneros" id="menu_library_yubo_genre"
34       parent="menu_library_yubo_management"
35       action="action_library_yubo_genre" sequence="40"/>
36   </data>
37 </odoo>

```

Loan.xml , **Views** de préstamos de libros

```
views > loan.xml > odoo > data > record > field > form > sheet > div > h1
1 <odoo>
2 <data>
3 <record model="ir.ui.view" id="view_library_yubo_loan_tree">
4 <field name="name">library.yubo.loan.tree</field>
5 <field name="model">library_yubo.loan</field>
6 <field name="arch" type="xml">
7 <tree decoration-info="duration &lt; 7" decoration-danger="duration &gt; 14">
8 <field name="book_id"/>
9 <field name="author_id"/>
10 <field name="member_id"/>
11 <field name="loan_date"/>
12 <field name="return_date"/>
13 <field name="duration" sum="Total Días"/>
14 </tree>
15 </field>
16 </record>
17
18 <record model="ir.ui.view" id="view_library_yubo_loan_form">
19 <field name="name">library.yubo.loan.form</field>
20 <field name="model">library_yubo.loan</field>
21 <field name="arch" type="xml">
22 <form>
23 <sheet>
24 <div class="oe_title">
25 <h1>
26 <field name="name" readonly="1"/>
27 </h1>
28 </div>
29 <group>
30 <group string="Información del Préstamo">
31 <field name="book_id" options="{ 'no_create': True }"/>
32 <field name="member_id"/>
33 </group>
34 <group string="Fechas y Duración">
35 <field name="loan_date"/>
36 <field name="return_date"/>
37 <field name="duration" readonly="1"/>
38 </group>
39 </group>
40 </sheet>
41 </form>
42 </field>
43 </record>
44
45 <record model="ir.actions.act_window" id="action_library_yubo_loan">
46 <field name="name">Préstamos</field>
47 <field name="res_model">library_yubo.loan</field>
48 <field name="view_mode">tree,form</field>
```

```
50
51 <menuitem name="Préstamos" id="menu_library_yubo_loan"
52 parent="menu_library_yubo_management"
53 action="action_library_yubo_loan"
54 sequence="20"/>
55 </data>
56 </odoo>
```

Member.xml , **Views** de socio que préstamos de libros

```
views > member.xml > odoo
1 <odoo>
2   <data>
3     <record id="view_partner_tree_library_inherit" model="ir.ui.view">
4       <field name="name">res.partner.tree.library.inherit</field>
5       <field name="model">res.partner</field>
6       <field name="inherit_id" ref="base.view_partner_tree"/>
7       <field name="arch" type="xml">
8         <xpath expr="//field[@name='display_name']" position="after">
9           <field name="member_number" optional="show"/>
10        </xpath>
11      </field>
12    </record>
13    <record id="view_partner_form_library_inherit" model="ir.ui.view">
14      <field name="name">res.partner.form.library.inherit</field>
15      <field name="model">res.partner</field>
16      <field name="inherit_id" ref="base.view_partner_form"/>
17      <field name="arch" type="xml">
18        <xpath expr="//field[@name='phone']" position="after">
19          <label for="member_number" string="Número de Socio" />
20          <field name="member_number" placeholder="Ej. SOCIO-001"/>
21        </xpath>
22      </field>
23    </record>
24
25    <record model="ir.actions.act_window" id="action_library_yubo_member">
26      <field name="name">Socios</field>
27      <field name="res_model">res.partner</field>
28      <field name="view_mode">tree,form</field>
29    </record>
30
31    <menuitem name="Socios" id="menu_library_yubo_member"
32      parent="menu_library_yubo_management"
33      action="action_library_yubo_member" sequence="50"/>
34  </data>
35 </odoo>
```

La captura muestra la configuración del archivo ir.model.access.csv, donde se vinculan los modelos técnicos de la base de datos (como model_library_yubo_book) con los permisos de usuario.

```
addons > library_yubo > security > ir.model.access.csv
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_library_yubo_book,library_yubo_book,model_library_yubo_book,base.group_user,1,1,1,1
3 access_library_yubo_author,library_yubo_author,model_library_yubo_author,base.group_user,1,1,1,1
4 access_library_yubo_genre,library_yubo_genre,model_library_yubo_genre,base.group_user,1,1,1,1
5 access_library_yubo_loan,library_yubo_loan,model_library_yubo_loan,base.group_user,1,1,1,1
```


Manifest.py , se ve views/book.xml carga primero

```

__manifest__.py
1  # -*- coding: utf-8 -*-
2  {
3      'name': "library_yubo",
4
5      'summary': "Short (1 phrase/line) summary of the module's purpose",
6
7      'description': """
8  Long description of module's purpose
9  """,
10
11     'author': "My Company",
12     'website': "https://www.yourcompany.com",
13
14     # Categories can be used to filter modules in modules listing
15     # Check https://github.com/odoo/odoo/blob/15.0/odoo/addons/base/data/ir_module_category_data.xml
16     # for the full list
17     'category': 'Uncategorized',
18     'version': '0.1',
19
20     # any module necessary for this one to work correctly
21     'depends': ['base', 'contacts'],
22
23     # always loaded
24     'data': [
25         'security/ir.model.access.csv',
26         'views/book.xml',
27         'views/author.xml',
28         'views/genre.xml',
29         'views/loan.xml',
30         'views/member.xml',
31     ],
32     # only loaded in demonstration mode
33     'demo': [
34         'demo/demo.xml',
35     ],
36 }

```

Api para lista todos los libros que tienes

```

controllers > controllers.py > LibraryYubo > get_books
1  # -*- coding: utf-8 -*-
2  from odoo import http
3  from odoo.http import request
4  import json
5
6  class LibraryYubo(http.Controller):
7
8      # 1. API: Obtener la lista completa de libros (GET)
9      # Dirección de acceso: http://localhost:8069/api/books
10     @http.route('/api/books', auth='public', type='http', methods=['GET'], csrf=False)
11     def get_books(self, **kw):
12         books = request.env['library_yubo.book'].sudo().search([])
13         book_list = []
14         for book in books:
15             book_list.append({
16                 'id': book.id,
17                 'title': book.name,
18                 'author': book.author_id.name,
19                 'state': book.state,
20             })
21
22         return request.make_response(
23             json.dumps({'status': 200, 'data': book_list}),
24             headers={'Content-Type': 'application/json'})
25

```

4.4 Pruebas de funcionamiento

Prueba de api

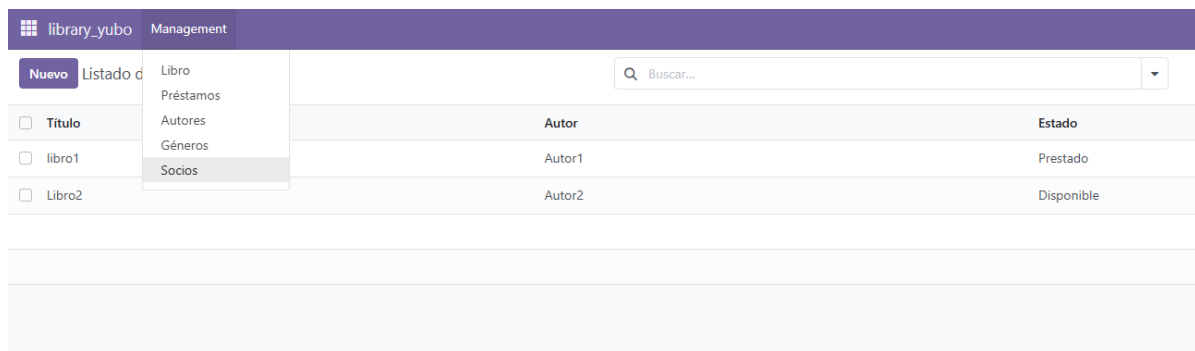
se ver solo tengo dos libros

<http://localhost:8069/api/books>

```
{
  "status": 200,
  "data": [
    {
      "id": 1,
      "title": "libro1",
      "author": "Autor1",
      "state": "borrowed"
    },
    {
      "id": 2,
      "title": "Libro2",
      "author": "Autor2",
      "state": "available"
    }
  ]
}
```

Prueba de aplicación

Basándonos en el código anterior ya podemos acceder a la interfaz de Odoo.



The screenshot shows the Odoo library management interface. At the top, there is a header bar with the logo and the text 'library_yubo Management'. Below this, there is a navigation menu with options: 'Nuevo', 'Listado de', 'Libro', 'Préstamos', 'Autores', 'Géneros', and 'Socios'. A search bar is located to the right of the menu. The main content area displays a table with the following data:

☐	Título	Autor	Estado
☐	libro1	Autor1	Prestado
☐	Libro2	Autor2	Disponible

Libros,

Nuevo Listado de Libro ⚙

Q Buscar...

☐ Título	Autor	Estado
☐ libro1	Autor1	Prestado
☐ Libro2	Autor2	Disponible

Dentro de libro tengo vista Kanban

library_yubo Management

Nuevo Listado de Libro ⚙

Q Buscar...

libro1 Autor: Autor1 Prestado	Libro2 Autor: Autor2 Disponible	Libro 3 Autor: Disponible	Libro 5 prueba Autor: Autor 3 prueba Disponible
--	--	--	--

Préstamos

library_yubo Management

Nuevo Préstamos ⚙

Q Buscar...

☐ Libro	Autor	Socio	Fecha de Préstamo	Fecha de Devolución
☐ libro1	Autor1	Ana Developer	11/01/2026	14/01/2026
☐ Libro2	Autor2	Azure Interior, Brandon Freeman	11/01/2026	13/01/2026

Autores

library_yubo Management

Nuevo Autores ⚙

Q Buscar...

☐ Nombre	Veces Prestado	Libros
☐ Autor1	1	libro1
☐ Autor2	1	Libro2

2

Género de libros

library_yubo Management

Nuevo

Géneros

☐ Género

☐ Aventura

☐ No se

Y Socios

library_yubo Management

Nuevo

Socios

Buscar...

☐ Número de Socio	Nombre	Teléfono	Correo electrónico	Comercial	Actividades
☐ SOCIO-001	usuario111				
☐ 1111	usuario2222				

4.5 Despliegue de la aplicación

El despliegue de este proyecto se ha realizado en un **servidor local**

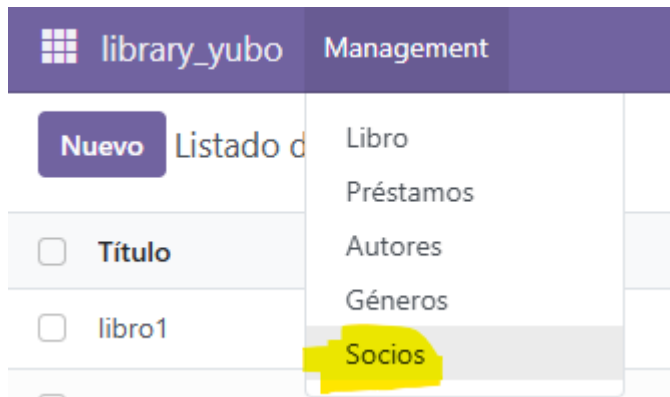
Para el despliegue, se han configurado tres servicios principales coordinados mediante un archivo docker-compose.yaml:

- Servidor Odoo: Contenedor que ejecuta la lógica de negocio de la biblioteca.
- Base de Datos PostgreSQL: Contenedor que almacena toda la información del catálogo, socios y préstamos.
- pgAdmin4: Herramienta web para la administración y supervisión de la base de datos SQL.

5 Manuales

5.1 Manual de usuario

En Management selecciona socio y dar botón nuevo



Datos importantes

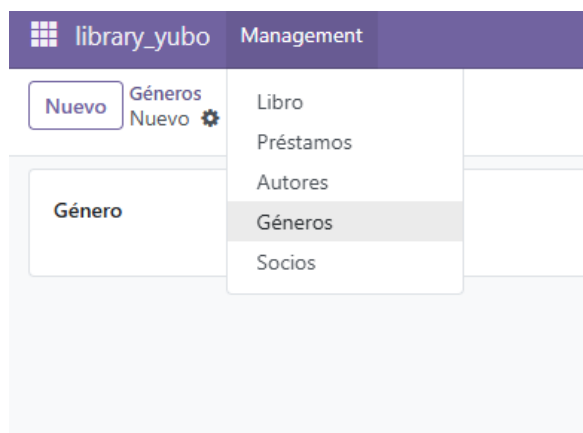
Nombre de socio y número de Socio

The screenshot shows the 'Nuevo' form for adding a new user. The 'Individuo' radio button is selected, and the name 'Usuario prueba' is entered. The form includes fields for 'Contacto', 'Calle...', 'Calle 2...', 'Ciudad', 'Provincia', 'C.P.', 'País', 'NIF?', 'Puesto de trabajo', 'Teléfono', 'Móvil', 'Correo electrónico', 'Sitio web', 'Título', and 'Etiquetas'. The 'Número de Socio' field is highlighted with a yellow background and contains the value '2222'. At the bottom, there are tabs for 'Contactos y direcciones', 'Ventas y compras', 'Facturación / Contabilidad', and 'Notas internas', and an 'Añadir' button.

Dar botón nuevo se ver , creado una usuario nuevo

Nuevo Socios		Buscar...				
☐	Número de Socio	Nombre	Teléfono	Correo electrónico	Comercial	Actividades
☐	222	Usuario prueba				○
☐	SOCIO-001	usuario111				○
☐	1111	usuario2222				○

En Management selecciona Géneros, también clic botón nuevo



Escribe género y clic botón nuevo



The screenshot shows the 'Géneros' management form. The 'Nuevo' button is highlighted in yellow. The 'Género' field is filled with the text 'Fantasía', which is also highlighted in yellow. The form includes a search icon and a refresh icon.

Se ver creado bien

Nuevo Géneros		Buscar	
☐	Género		
☐	Aventura		
☐	No se		
☐	Novela histórica.		
☐	Fantasía		

Parte libro

Nombre de autor y clic nuevo

library_yubo Management

Nuevo

Autores
Nuevo

Nombre

Autor 3 prueba

Libros Publicados

Título	Autor

Si después añadido un libro en libros publicados se ver que libro tienes

Nuevo

Autores
Autor2

Nombre

Autor2

Libros Publicados

Título	Autor	Estado
Libro2	Autor2	Disponible

Crear nuevo libro , rellena título, estado, Autor y Genero

Nuevo Listado de Libro
Nuevo ⚙️ 📁 ↺

Título	Libro 5 prueba
Estado	Disponible
Autor	Autor 3 prueba
Géneros	Género Añadir una línea

Autor puede seleccionar

Autor Autor 3 prueba

Géneros

- Autor1
- Autor2
- Autor 3 prueba
- Autor 4 prueba
- Buscar más...

Géneros puede clic Añadir una línea para seleccionar generos

Título	Libro 5 prueba
Estado	Disponible
Autor	Autor 3 prueba
Géneros	Género Añadir una línea

Añadir: Géneros

⚙️ 🔍 Buscar... ▼

- ☐ Género
- ☐ Aventura
- ☐ No se
- ☐ Novela histórica.
- ☐ Fantasía

Quando tienes todos , pues clic nuevo

Nuevo

Listado de Libro

Nuevo

Título

Libro 5 prueba

Estado

Disponible

Autor

Autor 3 prueba

Géneros

Género

Fantasía

Añadir una línea

Creado bien

library_yubo Management		
Nuevo Listado de Libro Buscar...		
☐ Título	Autor	Estado
☐ libro1	Autor1	Prestado
☐ Libro2	Autor2	Disponible
☐ Libro 3		Disponible
☐ Libro 5 prueba	Autor 3 prueba	Disponible

Y préstamos

NEW

INFORMACIÓN DEL PRÉSTAMO

FECHAS Y DURACIÓN

Libro

Socio

Fecha de Préstamo

Fecha de Devolución

Días de Préstamo

11/01/2026

0

Seleccionar Socio y fecha de devolución

Se calcular automáticamente día de préstamo

INFORMACIÓN DEL PRÉSTAMO		FECHAS Y DURACIÓN	
Libro	Libro 5 prueba	Fecha de Préstamo	11/01/2026
Socio	Usuario prueba	<u>Fecha de Devolución</u>	<u>14/01/2026</u>
		Días de Préstamo	3

Clic botón nuevo crear y creado préstamos

library_yubo Management					
Nuevo Préstamos		Buscar...		1-3 / 3	
☐ Libro	Autor	Socio	Fecha de Préstamo	Fecha de Devolución	Días de Préstamo
☐ libro1	Autor1	Ana Developer	11/01/2026	14/01/2026	3
☐ Libro2	Autor2	Azure Interior, Brandon Freeman	11/01/2026	13/01/2026	2
☐ Libro 5 prueba	Autor 3 prueba	Usuario prueba	11/01/2026	14/01/2026	3

5.2 Manual de instalación

- Copiar la carpeta library_yubo en el volumen de addons.
- Ejecutar docker-compose restart.
- En Odoo, activar "Modo Desarrollador" y pulsar "Actualizar lista de aplicaciones".
- Buscar "Library Yubo" e instalar.

6 Conclusiones y posibles ampliaciones

El desarrollo de este proyecto ha permitido profundizar en el funcionamiento interno de Odoo. Se han cumplido todos los requisitos, incluyendo el uso de ORM avanzado (env, search), campos computados, herencia de vistas y la creación de una API REST funcional. La estructura modular del código facilita su mantenimiento y futura expansión.

7 Bibliografía

Apuntes de profesor :

[2 DAM SGE UD 5 Desarrollo modulos.pdf](#)

[2 DAM SGE UD 6 Desarrollo modulos conceptos avanzados.pdf](#)

Odoo S.A. (2025). Odoo Developer Documentation. Recuperado de:

<https://www.odoo.com/documentation/16.0/>